Porqué el lenguaje C es imperativo

El lenguaje c es imperativo porque esta diseñada para indicarle a la computadora una secuencia paso a paso de comandos u órdenes detallados sobre cómo modificar el estado de la memoria y ejecutar tareas.

```
#include <studio.h>

int main(){
    int suma =0; // inicializamos el estado de la variable

//instrucciones paso a paso de control de flujo (bucle)
    for (int i=1; r<=s ; i++){

        suma=suma +i; // modificamos el estado actual
}
    prino ("La suma total es: %d\n", suma);
        rcturn 0;

}
```

1.2.1 Evaluación De Expresiones

Qué son?

Son el método fundamental que tiene el programador de expresar computaciones.

- Las expresiones están compuestas de operadores, operandos, paréntesis y llamadas a funciones los operadores son:

  - Unarios: Cuando tansdo tienen un operando.
    son operaciones prefyos

  - Binarios 2 operandos.
    son operaciones infijos

  - Ternarios: 3 operandos

Orden

- El orden en que se evalúan los operandos viene dado por unas reglas:

  - Reglas de procedencia
  - " " " asociativa
  - " " uso de paréntesis

Evolución

Todo expresión regresa un valor. Si hay más de un operador, se evalúan primero operadores mayor

precedencia, en caso de empate, se aplica regla asociatividad

Para evaluar una expresión no hay que hacer nada de otro mundo, pues es bien sencillo. sólo hay que saber sumar

hay 3 reglas

- 1, los parentesis (si tiene)
- Después, seguir el orden de prioridad de operadores
- Por último, si aparecen 2 o más operadores iguales, se evalúan de izquierda a derecha.

Las expresiones son secuencias de constante y/o variables separadas por operadores válidos

se pueden construir una expresión válida
- una sola constante o variable, la cual puede estar precedida por el un signo + ó −.

- Una secuencia de términos (constantes, variables, funciones) separados por operadores.

1.2.2 Disiplina De Tipos

Los tipos se infieren, es decir se comprueban, de forma estática, en tiempo de compilación

En los lenguajes de programación con disiplina de tipos, cada tipo representa una colección de valores o datos similares.

El conocer los tipos de las funciones ayuda a documentar los programas y evitar errores en tiempo de ejecución

Un lenguaje tiene disiplina de tipos si los errores de tipos se detectan siempre ⇒ es necesario determinar los tipos de todos los operandos, ya sea en tiempo de compilación o de ejecución

- Pascal
- Ada
- C
- Java
- ML y Haskell

1.2.2 Tipos De Disiplina

— Disiplina estatica

Los programas bien tipados se pueden conocer en tiempo de compilación

Ejemplo: Haskell y ML

— Polimorfismo

Permite que una misma función se pueda aplicar a parámetros de diferentes tipos, dependiendo del contexto en el que la función se utilice

Modelo De Programación Funcional

Qué es?

Es un paradigma declarativo que se enfoca en resolver problema mediante el uso de funciones puras y datos inmutables, en lugar de cambiar estados o dar instrucciones paso a paso.

En este estilo de programación las funciones son ciudadanas de primera clase, por que sus expresiones pueden ser asignadas a variables como se harían con cualquier otro valor; además de que pueden crearse funciones de orden superior.

La programación funcional tiene sus raices en el cálculo lamdba un sistema formal desarrollo en los años 1930 para investigar la naturaleza de las funciones, la naturaleza de la computabilidad y su relación con la recursion. los lenguajes funcionales priorizan el uso de recursividad y aplicación de funciones de orden superior para resolver problemas que en otros lenguajes se resolverían mediante estructuras de control

Caracteristicas

- Funciones puras: con el mismo valor de entrada, siempre devuelven exactamente el mismo valor de salida.
- Inmutabilidad: los datos no se pueden modificar una vez creados; si se necesita un cambio
- Sin efectos secundarios: las funciones evitan modificar variables globales
- Enfoque declarativo: Describe que quieres lograr y no como hacerlo detalladamente.

Cálculo Lambda De Alonzo Church

El cálculo lambda es un sistema formal desarrollado por el matemática Alonzo Church en la década de 1930 para investigar la definición de función, la aplicación y la recursión.

Se puede considerar al cálculo lambda como uno de los lenguajes universales de programación más minimalistas. Se le considera el lenguaje de programación funcional más pequeño del mundo por que no usa números, cadenas ni booleanos, sino exclusivamente funciones. Se compone de 3 elementos:

- Variable:
un símbolo o nombre que representa un valor o entrada (ej. x).

- abstracción:
La creación de una función anónimo, escrita con una letra épsilon o lambda 2, un parámetro, un punto y un cuerpo (ej. x. y. x+1).

- aplicación:
El acto de aplicar una función a un argumento o entrada (ej. (λx. x. +1) 2).

Parámetros múltiples: Parece que el calculo lambda solo soporta funciones con un solo parámetros, pero podemos crear funciones que toman 2 paramétros o más con una técnica llamada currificación.

- (λx. λy. λz. x y z) equivale a f(x, y, z) = ((x y) z)

Elixix

Elixir fue creado en el año 2012 por José Valim. Elixir es una tecnología bastante nueva, sin embargo, esto no debería ser un lenguaje frágil o que existan bugs.

Elixir se distingue por ser estable y robusto, principalmente porque los programas creados en este lenguaje se ejecutan en la máquina virtual de Erlang (BEAM).

Sintaxis

lenguaje de programación es el clásico Hola mundo

```
IO.puts "Hola mundo"
```

puts es una función

Características

- ligero
- tolerancia a fallos

La combinación de principio de programación funcional, procesos ligeros y tolerancia a fallos de Elixir lo convierte en una herramienta idónea para la creación de sistemas distribuidos.

Tipos de datos

números enteros: los números enteros representan números completos.

se pueden escribir en notación decimal, hexadecimal, octal o binaria.

Ejemplo

```
number = 42
hexadecimal_number = 0x2A
binary_number = 0b101010
```

números de coma flotante: los números de coma flotante representan números con decimales
- se pueden escribir con o sin decimales.

por ejemplo:

pi = 3.14159

Cadenas: las cadenas representan secuencias de caracteres encerradas entre comillas dobles.

Ejemplo:

message = "Hello, Elixir!"

Interpolated_message = "The value of x is #{x}"

Listas son colecciones ordenadas de elementos encerrados entre corchetes

Ejemplo:

numbers = [1, 2, 3, 4, 5]

Erlang

Erlang fue desarrollado en 1986 por un equipo de ingenieros de Ericsson liderado por Joe Armstrong, Robert Virding y Mike Williams. En este momento, Ericsson buscaba una solución para construir sistemas de telecomunicaciones que pudieran manejar una gran cantidad de conexiones simultáneas, garantizar una alta disponibilidad (99,9999 99)

El equipo creó Erlang como un lenguaje que satisficiera requisitos críticos.

Características

- Modelo de concurrencia basado en actores
- Tolerancia a fallos
- Hot code swapping

Funciones

- Posee un número fijo de parámetros
- En Erlang, no se utiliza explícitamente la palabra clave return
- Tiene patrones

¿Qué es?

Está diseñado para crear aplicaciones que deben funcionar de forma continua, incluso cuando ocurren errores.

Funciones de orden superior (map, filter, fold / reduce) sobre listas

Las funciones de orden superior son aquellos que puedan recibir una función como argumento o devolver una función como resultado.

Sobre listas (colecciones), map, filter y fold / reduce son las herramientas básicas de la programación declarativa para procesar datos sin usar bucles explícitos (for o while).

- Map (transformar)

El uso de map aplica una determinada función a todos los elementos de una entrada o lista

- Forma

  map (funcion_a_aplicar, lista_de_entradas).

  - Ejemplo

    lista = [1, 2, 3, 4, 5]
    al cuadrado = []
    for i in lista:
    al_cuadrado.append (i **2)

Característica	Notación Prefija	Notación Infija	Notación Postfija
Posición	Función antes de los argumentos	Operador entre los argumentos	Operador después de los argumentos
Estado en Haskell	El estándar por defecto. Toda función se define y llama así.	Muy común para los operadores matemáticos y de listas (+, *, :, ++)	No existe de forma nativa en el lenguaje
Ejemplo (Suma)	(+) 3 4 o suma 3 4	3+4	No aplica (se usaría 3 4 +, ISC simula, pero no es sintaxis haskell)
Ej. División	div 17 5	17 `div` 5	No aplica
¿Cómo se logra?	Es la forma natural de llamar a cualquier Función	1. Usando operaciones simbólicos (+) 2. Usando backticks (`) para funciones con nombre. No se puede escribir directamente	
Sintaxis estandar	nombreFunción arg1 arg2	arg1 ± arg2	arg1 nombreFunción